The real reasons Claude got dumber

Transcript

0:00
Last week, I did some deep coverage on
0:02
some Anthropic issues. In particular, it
0:04
seemed like Claude was behaving much
0:05
dumber. Thankfully, for once, Anthropic
0:07
actually came out and admitted that that
0:09
did happen. But they admitted this
0:10
multiple days later after confirming
0:12
that this outage had occurred for weeks,
0:14
three times. There were three separate
0:17
major issues that compiled on top of
0:18
each other that made most anthropic
0:20
models a lot dumber a lot of the time.
0:22
And they finally decided that hiding all
0:24
of the details behind the scenes might
0:26
not be the best call. I complained about
0:28
this a ton in my video last week and I'm
0:31
very thankful to say Anthropic has
0:33
responded. They actually took the time

0:34
to put up an article discussing the
0:37
three recent issues that occurred
0:38
causing clawed models to be way dumber.
0:41
This is a level of transparency I've
0:43
never seen from anthropic. Usually their
0:45
transparency is limited to their
0:47
research and their benchmarks for
0:49
different like misalignment tests. I've
0:51
never seen them go this ind depth on
0:52
their own infra on what problems they
0:54
have behind the scenes and actually
0:56
talking about the work that the
0:57
engineers are doing and what failed and
0:59
more importantly why. To be very very
1:02
clear, it does seem like the clawed
1:03
models are now back to behaving as
1:05
expected. That doesn't mean you're
1:07
getting a refund, which is annoying. And
1:08
it doesn't mean they won't have issues
1:09
again in the future, which is scary. I'm
1:12
thankful to have actual information on
1:14
what went wrong here, especially

1:15
considering the severity of this bug and
1:17
how long it was going for. All of that
1:19
said, I'm not getting a refund either,
1:20
and someone's got to cover my cloud code
1:22
bills. So, quick word from today's
1:23
sponsor, and then we'll dive in. There's
1:25
a fundamental problem with most of the
1:27
AI coding tools we use today. They're
1:29
being built on the inside, not the
1:30
outside. What the heck do I even mean?
1:33
Well, most of the tools that we use,
1:34
things like cursor, cloud code, and
1:36
more, are built inside of existing
1:38
things. Cursor is built inside of VS
1:40
Code. Cloud code is built inside of the
1:42
terminal. They're limited by the shell
1:44
that they're built within. What if they
1:46
weren't? What if someone rethought the
1:48
whole shell to build a better experience
1:50
from the outside so they could make
1:51
something truly incredible? Well, we
1:53
have an answer and it's today's sponsor,

1:55
Warp. They built an all-in-one agentic
1:57
development environment. Everything from
1:59
the terminal to the code tools to so
2:01
much more. The terminal experience in
2:02
and of itself is magical. Just having
2:04
good autocomplete and history management
2:06
in a terminal is so cool. Then you press
2:08
command I and now you can ask it to do
2:11
things to the code. You can attach
2:12
context with a hotkey. You can navigate
2:14
everything with hotkeys. It's really
2:16
focused on power users and developers,
2:18
you know, us. And it feels fundamentally
2:21
different as a result. You never feel
2:22
like this is being stuffed in somewhere.
2:25
You feel like the whole tool is built to
2:26
be a good experience for this way of
2:28
building. This is a real project I spun
2:30
up using warp. This is a real prompt
2:32
that I sent asking it to do my classic
2:34
image generation tool. And the results
2:36
are stunning because the results are all

2:39
using GPT5. I don't have to use GPT5.
2:42
You have access to all the major models
2:43
you'd ever wanted more. But the fact
2:45
that you get 150 free requests with any
2:47
of those models is a pretty good deal in
2:49
and of itself. It's still so cool. I can
2:51
just ask it whatever and if it needs to
2:52
run a command, it will. And if it
2:54
doesn't, it won't. Obviously, it knows
2:55
modern tools like Nex.js and Tailwind,
2:57
but if you're using things it doesn't,
2:58
you can add that to the rules or the MCP
3:00
servers that are global for the entire
3:02
thing. So, you can make it generate and
3:04
run bash commands based on things that
3:06
it learned about MCP server. That's so
3:08
useful and makes MCP actually feel
3:10
valuable to me. There's a reason so many
3:12
engineers are making the move to Warp.
3:14
just makes you more productive. Check
3:16
them out now at soy.link/warp and get
3:18
your first month for 70% off with code

3:20
theo. A post-mortem of three recent
3:22
issues. This is a technical report on
3:25
three bugs that intermittently degraded
3:27
responses from Claude. Below we explain
3:29
what happened, why it took time to fix,
3:31
and what we're changing. Between August
3:33
and early September, three
3:34
infrastructure bugs intermittently
3:36
degraded Claude's response quality.
3:38
We've now resolved these issues and want
3:40
to explain what happened. I'm sure
3:41
there's no particular reason why you
3:43
want to do that now. Definitely not. In
3:46
early August, a number of users began
3:48
reporting degraded responses from
3:49
Claude. These initial reports were
3:50
difficult to distinguish from normal
3:52
variation in user feedback. By late
3:54
August, the increasing frequency and
3:56
persistence of these reports prompted us
3:58
to open an investigation that led us to
4:00
uncover three separate infer bugs. Let's

4:02
be clear about this. The bugs started in
4:06
early August. They assumed that they
4:08
were all just bad feedback and they
4:10
didn't have any eval or tooling or
4:12
checks that could confirm the
4:15
performance had degraded. I I know LLMs
4:18
are non-deterministic. I'm very aware of
4:21
the fact that you can't really guarantee
4:23
a result with any of these things, but I
4:25
would hope you're running enough tests
4:28
in the background about just random
4:30
checks, maybe run 10 a day, just seeing
4:33
what the outputs look like and seeing if
4:34
the regularity of correct responses goes
4:37
down. Did they really not have anyone
4:38
running benches internally that they
4:40
noticed a degradation from? Because this
4:43
is scary that it went on for almost a
4:45
month that early August they mostly
4:47
ignored it and then by late August the
4:49
increasing frequency prompted them to
4:51
look into it more in depth. To state it

4:53
plainly, we never reduce model quality
4:55
due to demand, time of day or server
4:57
load. They do inadvertently reduce model
4:59
quality in order to give their
5:00
researchers more GPUs, which we'll talk
5:02
about throughout this, but the problems
5:03
that users reported in this case were
5:05
due to infrastructure bugs alone. The
5:07
reason they were making the infer
5:09
changes that caused these bugs is
5:10
definitely worth going into, but I will
5:12
say upfront some of my previous
5:14
assumptions in my last video were wrong.
5:16
I had pretty confidently asserted that
5:18
it seemed like their attempts to fix it
5:20
cause more bugs. That does not appear to
5:22
be the case and we'll go in depth on
5:23
that momentarily. We recognize users
5:25
expect consistent quality from Claude
5:27
and we maintain an extremely high bar
5:28
for ensuring that infrastructure changes
5:30
don't affect model outputs. In these

5:32
recent incidents, we didn't meet that
5:34
bar. Following postmortem explains what
5:36
went wrong, why detection and resolution
5:38
took longer than we would have wanted,
5:40
and what we're changing to prevent
5:41
similar future incidents. We don't
5:43
typically share this level of technical
5:44
detail about our infra. Yeah, no
5:46
But the scope of complexity of these
5:48
issues justified a more comprehensive
5:49
explanation. I agree. Thank you for
5:51
doing this. I hope it sets a trend.
5:53
There's no reason to hide these details.
5:55
It doesn't cost you anything to share
5:57
this information. I don't know why you
5:59
waited so long to do it. It almost feels
6:01
like you were waiting until the
6:02
investment round closed.
6:04
Like seriously, it just it doesn't make
6:06
sense to hide this information.
6:08
How we serve Claude at scale. We serve
6:10
Claude to millions of users via our

6:12
first party API, Amazon Bedrock and
6:14
Google Cloud's Vert.ex AI. We deploy
6:16
Claude across multiple hardware
6:18
platforms, namely AWS Tranium, Nvidia
6:21
GPUs, and Google's TPUs. This approach
6:23
provides the capacity and geographic
6:25
distribution necessary to serve users
6:27
worldwide. Each hardware platform has
6:30
different characteristics and requires
6:31
specific optimizations. Despite these
6:34
variations, we have strict equivalent
6:36
standards for model implementations. Our
6:38
aim is that users should get the same
6:40
quality responses regardless of which
6:42
platform serves their request. This is
6:44
particularly funny because I find that
6:46
all of the other platforms are slightly
6:47
more reliable than Anthropics,
6:49
especially after all of this. You're not
6:51
already familiar with Open Router and
6:53
you're watching this video, you should
6:54
be familiar with Open Router. They show

6:56
all the different providers that you can
6:58
use Claude Sonnet 4 with, for example.
7:00
And here we can see that Bedrock's
7:01
actually quite a bit faster, has really
7:03
good uptime. Now, it wasn't for a bit,
7:05
but they've seem to have fixed it on
7:06
Bedrock side. Google Vertex has pretty
7:09
solid throughput, also really good
7:10
uptime. And then Enthropic has the
7:13
slowest speeds of any of the providers.
7:14
And right now, their uptime is decent,
7:16
but that is not always a guarantee. Even
7:19
just two days ago, it was at 99.6%.
7:21
Which is kind of scary.
7:24
Yeah, uptime is far from a guarantee.
7:27
What's cool with open router is it'll
7:29
automatically bounce your traffic
7:30
between the different providers
7:31
depending on what ones are most or least
7:33
reliable at a given time. They all cost
7:35
the same, so who cares. That said, when
7:38
Anthropic's having issues, you can hop

7:39
to the other providers, which likely
7:40
aren't. Back to their coverage, cuz
7:43
there's fun info here. In particular,
7:45
last sentence. This complexity means
7:47
that any infra changes require careful
7:49
validation across all platforms and
7:51
configurations. Supposedly, that means
7:54
that these changes actually did
7:55
negatively affect the other platforms. I
7:57
don't know for sure, but it seems not
7:59
unlikely. Here is the timeline as they
8:02
describe it. Yellow is degraded, red is
8:05
and green is fixed for the three
8:08
different issues that they had. Starting
8:10
August 5th, they had a context window
8:12
routing error which affected around 1%
8:14
of Sonnet 4 requests. That's brutal.
8:17
that there's a one in a 100 chance that
8:19
your request gets routed to the wrong
8:21
place and it performs worse. The reason
8:23
for this was the recent introduction of
8:25
the 1 million token context windows in

8:27
cloud models. If you haven't paid much
8:29
attention to the performance of models
8:31
when you increase token amounts and when
8:33
you increase context size, the
8:34
performance tends to go down very quick.
8:36
It's called the needle in the haststack
8:38
problem, but it goes much further than
8:39
that. When you give a model more
8:40
context, the quality of its outputs goes
8:42
down. My assumption here is that the 1
8:45
million token version of Claw Force on
8:47
it probably had less than ideal
8:50
performance in general. Like the changes
8:52
they made to allow it to handle a
8:53
million tokens made it dumber in
8:55
general. And the bug as they detail here
8:57
is that it would send traffic to that
8:59
version when it shouldn't have, even if
9:01
it was well under a million tokens. Then
9:03
on August 25th, the output corruption
9:06
error started. On August 26th, they had
9:08
a top K like code hack they did that

9:12
broke a bunch of stuff. August 29th,
9:14
they made load balancer changes that
9:16
caused 16% of Sonnet 4 requests to get
9:18
routed incorrectly. And from August 29th
9:21
to September 4th, those problems were
9:23
persistent. They did manage to fix the
9:25
corrupted output issues a little earlier
9:27
on September 2nd, but then this other
9:30
bug took until September 12th. So, it
9:32
took them a while to fix these things.
9:35
Let's talk about those different bugs.
9:37
The overlapping nature of these bugs
9:38
made diagnosis particularly challenging.
9:40
The first bug was introduced on August
9:41
5th, affecting approximately 1% of
9:43
requests made to Sonet 4. Two more bugs
9:45
arose from deployments on August 25th
9:47
and 26th. Although initial impacts were
9:50
limited, a load balancing change on
9:51
August 29th started to increase affected
9:53
traffic. This caused many more users to
9:55
experience issues while others continued

9:57
to see normal performance creating
9:59
confusion and contradictory reports.
10:02
Three overlapping issues. First one is
10:04
the context window routing error. In
10:06
August 5th, some sonnet requests were
10:07
misouted to servers configured for the
10:09
upcoming 1 million token context window.
10:11
This bug initially only affected 1% of
10:13
requests. But on August 29th, a routine
10:15
load balancing change unintentionally
10:16
increased the number of short context
10:18
requests being routed the 1 million
10:20
context servers. At the worst impacted
10:22
hour on August 31st, 16% of Sonor
10:25
requests were affected. Approximately
10:27
30% of cloud code users who made
10:29
requests during this period had at least
10:31
one message routed to the wrong server
10:33
type resulting in degraded responses. On
10:35
Amazon Bedrock, misouted traffic peaked
10:38
at2% of all sonnet for requests and
10:40
incorrected routing affected less than

10:42
0.004%
10:44
of requests on Google Cloud's Vert.ex AI
10:46
between August 27th and September 16th.
10:48
Again, it did affect the other
10:50
providers, but significantly less than
10:51
Enthropic. No conspiracy theories just
10:54
yet. We'll go into my conspiracies soon.
10:56
It is of note that some users were
10:58
affected more severely. I'm almost
10:59
certain I was considering how bad of an
11:01
experience I was having at the time. As
11:03
our routing is sticky, that meant that
11:05
once a request was served by the
11:07
incorrect server, subsequent follow-ups
11:09
were likely to be served by the same
11:11
incorrect server. This is so that if
11:13
you're caching context, you don't have
11:14
to wait as long for it to load. So when
11:16
I send a message, get a response, and
11:17
then send another, it's more likely that
11:19
you're hitting the same server. So, it's
11:20
more likely that it already has the

11:22
context cached for the things you just
11:23
said before and it can continue
11:24
accordingly. That's a good thing. It's
11:26
not good if the server you hit is bad
11:28
and now you're stuck to the bad server.
11:30
Resolution. We fixed the routing logic
11:32
to ensure short and long context
11:33
requests were directed to the correct
11:35
server pools. We deployed the fix on
11:37
September 4th. Roll out to our first
11:39
party platform as well as Google Cloud's
11:41
Vert.ex AI was completed by the 16th and
11:43
AWS Bedrock by the 18th.
11:46
Issue number two, output corruption. On
11:48
August 25th, we deployed a
11:50
misconfiguration to the Claude API TPU
11:52
servers that caused an error during
11:54
token generation. An issue caused by a
11:56
runtime performance optimization
11:58
occasionally assigned a high probability
12:00
to tokens that should rarely be produced
12:02
given the context. For example,

12:04
producing Thai or Chinese characters in
12:06
response to English prompts or producing
12:08
obvious syntax errors in code. If you
12:10
don't already understand how LLM
12:12
generate stuff, I'll give a really,
12:14
really, really brief and not great
12:16
overview. So, I take a prompt like, I
12:18
don't know, generate solutions for
12:20
advent of code in four different
12:21
programming languages.
12:24
You'll see that this is split into 13
12:26
tokens. A token is the way that your
12:30
text is broken up. Both your text that
12:32
you give to the model and the model's
12:34
generated output in order for it to map
12:36
all of these things in this giant
12:38
chaotic web of parameters.
12:44
Notice that I just added a word cuz it
12:45
spelled advent of code wrong. It did
12:47
adventive code, not advent of code. And
12:50
even though I changed this to be advent
12:52
of code, it's still the same number of

12:54
tokens. They're almost like kind of like
12:56
syllables, but it's not that clear a
12:57
split. Now we delete the space at the
12:58
end. We're add to 12. Cool. And when it
13:01
outputs things, the same thing happens.
13:03
If we take some code that was generated,
13:06
throw this at a model that's reliable.
13:08
Grock 4 fast.
13:16
Not very fast anymore. Let's throw this
13:18
in the tokenizer. And you'll notice it's
13:21
split up a hell of a lot more because
13:23
source code has a lot of tokens because
13:26
it needs to be able to break it up in a
13:28
way where it can generate the next
13:29
piece. So the way the model works is it
13:31
has all of these as parameters often
13:33
billions if not trillions of parameters.
13:36
So if we have words like the cat,
13:40
elephant, you pass it an input and then
13:42
it picks what it thinks the most likely
13:44
token is to respond with. This is a
13:47
parameter that it has inside of its

13:48
crazy web of data. And then it has
13:51
mappings to all sorts of other tokens
13:54
throughout that are weighted. So each of
13:57
these potential paths has like a number
13:59
0.5
14:01
08
14:03
0.2.
14:05
And all of the other tokens in the
14:07
current generation change the weight of
14:10
which points where. You effectively are
14:13
traversing through this based on what it
14:15
thinks is most likely based on the
14:18
previous terms. So the cat sits on
14:21
elephant for example from the the most
14:24
likely next thing was cat 0.8
14:26
We have cat routing to all sorts of
14:28
different things, but it thinks the most
14:29
likely to go to sits on elephant. And
14:31
that's really really roughly how the
14:34
generation happens. And all of these
14:37
weights are determined with math that
14:39
determines based on the previous inputs

14:42
what direction the text generation is
14:44
most likely to go in. So the problem
14:47
here was that the probability
14:49
calculation where it would determine
14:50
what the next token should be had some
14:52
bad math in it. And that corruption
14:54
resulted in certain unlikely tokens to
14:57
be snuck into your responses. So in the
15:00
middle of your English response to an
15:02
English question, you might just get
15:03
these random characters in the middle of
15:05
the response. This corruption affected
15:07
requests made to Opus 4.1 and 4 on
15:10
August 25th through 28th and requests to
15:12
Sonnet 4 from August 25th to September
15:15
2nd. Third party platforms were not
15:17
affected by this issue. That is a very
15:19
important detail here because now we're
15:22
going to start getting into some
15:23
conspiracies. All of the changes
15:25
Anthropic was making other than the
15:27
million token one, everything else they

15:29
did was with a very specific goal in
15:30
mind. Reduce compute per request. The
15:34
easiest way to reduce the amount of
15:35
compute a request takes is to make the
15:38
actual model dumber and work cheaper.
15:41
Ideally, you don't do that. And ideally,
15:43
Anthropic doesn't either. And I want to
15:44
be very clear, they did not intend to
15:46
make the model dumber. They did intend
15:49
to reduce the compute that was necessary
15:50
per request. And here is where the
15:52
conspiracy comes in. I talked about this
15:54
a bit in other videos about Enthropic.
15:56
They need more GPUs for the research
15:57
side of things. The research team, the
15:59
model training team, the data teams, all
16:01
the other teams at Enthropic are
16:03
competing with us as developers building
16:05
on the API for GPUs. Right now,
16:08
Enthropic was hoping to reduce the
16:10
amount of GPUs necessary to handle the
16:13
load from the API. If they can find a a

16:15
win that's like a 1% win in the compute
16:18
necessary for a request, that's a lot of
16:21
GPU they just freed up for their own
16:23
research. And that is very clearly one
16:25
of their specific pointed goals. How can
16:27
we free up GPUs for the work that we are
16:29
doing at whatever cost? And when you see
16:32
how many changes they're making, which
16:33
is why this timeline got so chaotic, you
16:35
can see just how hard they are trying to
16:37
do this. at every corner they can.
16:39
They're finding ways to cut the amount
16:40
of compute necessary in hopes of freeing
16:43
up more allocation that they can then
16:44
use for their research. That's also why
16:47
a lot of these changes were kind of
16:48
rushed out. I think that'll get extra
16:50
clear with issue three here, the
16:52
approximate top K XLAT TPU miso
16:55
compilation. On August 25th, we deployed
16:57
code to improve how Claude selects
16:59
tokens during text generation. This

17:01
change inadvertently triggered a latent
17:03
bug in the XLAT TPU compiler which has
17:05
been confirmed to affect requests to
17:06
Claude Haiku 3.5. We also believe this
17:09
could have impacted a subset of Sonnet 4
17:11
and Opus 3 on the cloud API. Third party
17:13
platforms were not affected by this
17:15
issue. They actually broke down this bug
17:17
more in detail. To illustrate the
17:18
complexity of these issues, here's how
17:20
the XLA compiler bug manifested and why
17:22
it proved particularly challenging to
17:23
diagnose. When claw generates text, it
17:25
calculates probabilities for each
17:26
possible next word, then randomly
17:28
chooses a sampling from the probability
17:30
distribution. We use top p sampling to
17:33
avoid nonsensical outputs, only
17:35
considering words whose cumulative
17:36
probabilities reach a threshold like.99
17:39
or.999. On TPUs, our models run across
17:42
multiple chips with probability

17:44
calculations happening in different
17:45
locations. To sort these probabilities,
17:47
we need to coordinate data between
17:49
chips, which is complex.
17:52
The problem of having multiple different
17:54
GPUs solving the same request at the
17:56
same time is absurd. There there's so
17:58
much history there, especially with the
18:00
chaos of the like import bands for
18:03
China. Maybe a video another time.
18:06
Absolute chaos there, though. And that's
18:07
the layer where these bugs were starting
18:09
to happen. In December of 2024, we
18:11
discovered that our TPU implementation
18:13
would occasionally drop the most
18:15
probable token when temperature was
18:17
zero. We deployed a workaround to fix
18:19
this case. You know the classic divided
18:21
by zero. This is a hack for top P 0 aka
18:25
temperature is zero. Due to oddjax
18:27
issues, it's possible that probs greater
18:29
than props max 1. It's all false. So we

18:32
manually always preserve the top logit
18:34
in each row. This is the math that they
18:37
did to handle when you have temperature
18:39
all the way down. Temperature is a
18:40
parameter that controls the randomness
18:42
of a model's predictions during text
18:44
generation. Higher temperature leads to
18:46
more creative and diverse outputs,
18:47
allowing for multiple variations in
18:49
phrasing and in the case of fiction,
18:51
variations in answers as well. Lower
18:54
temperatures result in more conservative
18:55
and deterministic outputs that stick to
18:57
the most probable phrasing and answers.
18:59
Just the temperature enables users to
19:00
encourage a language model to explore
19:02
rare, uncommon, or surprising word
19:04
choices and sequences rather than only
19:06
selecting the most likely predictions.
19:08
Again, with my little diagram here, if
19:10
we're following the tokens down the
19:12
different paths that are the most

19:13
likely, you end up with something kind
19:15
of deterministic because it's going down
19:18
the highest number for each thing. The
19:21
gap here barely matters. Actually, the
19:23
second one's slightly bigger. So, let's
19:24
um
19:26
99989 versus 99987.
19:29
If you have the temperature turned all
19:30
the way down to zero or you don't have a
19:32
concept of temperature, you just always
19:33
pick the best one. But temperature is
19:36
effectively letting you randomize and
19:37
say, "No, it's okay to pick something
19:39
really close, which lets you get more
19:41
variation in your outputs." Most model
19:43
providers have the top P or the
19:44
temperature set as a 1.0 or 0.8.
19:48
Usually, just the number that determines
19:51
how much wiggle room to give when it's
19:54
doing this math. And the mistake they
19:56
made is that their math here appears to
19:58
be broken. The root cause involve mixed

20:01
precision arithmetic. Our models compute
20:03
next token probabilities in BF16 which
20:06
is a 16- bit floatingoint number.
20:08
However, the vector processor is an FP32
20:10
native which is a 32-bit float. The CPU
20:13
compiler which is XLA can optimize
20:15
runtime by converting some operations to
20:18
FP32. Its optimization pass is guarded
20:20
by the XLA allow access precision flag
20:23
which defaults to true. This caused a
20:25
mismatch. Operations that should have
20:27
agreed on the highest probability token
20:29
were running at different precision
20:31
levels. The precision mismatch meant
20:33
that they didn't agree on which token
20:34
had the highest probability. This caused
20:36
the highest probability token to
20:38
sometimes disappear from consideration
20:40
entirely.
20:41
Fun. On August 26th, we deployed a
20:44
rewrite of our sampling code to fix the
20:46
precision issues and improve how we

20:47
handle probabilities and at the limit
20:50
that reach the top P threshold. But in
20:52
fixing these problems, we exposed a
20:53
trickier one. Here's their new test file
20:59
code snippet showing a minimize
21:00
reproducer emerged as part of the August
21:01
11th change that root caused the bug
21:03
being worked around in December 2024. In
21:05
reality, it's expected behavior of the
21:07
XLA allow access precision flag. Our fix
21:09
removed the December workound because we
21:11
believe we'd solved the root cause. This
21:13
led to a deeper bug in the approximate
21:14
top K operation performance optimization
21:17
that quickly finds the highest
21:18
probability tokens. The approximation
21:20
sometimes returned completely wrong
21:22
results, but only for certain batch
21:24
sizes and model configs. December
21:26
workound had been inadvertently masking
21:28
the problem. This is very clearly a
21:30
Slack screenshot that they just snuck in

21:32
here. They have it named accordingly.
21:34
Nope. Are they running this on Versel?
21:37
Nope. It's Google Cloud, but they are
21:39
using Next. Sorry, get nerdy about those
21:41
details. The bug's behavior was
21:43
frustratingly inconsistent. It changed
21:45
depending on unrelated factors such as
21:46
what operations ran before or after and
21:49
whether debugging tools were enabled.
21:51
Those are the best bugs when you turn on
21:53
debugging and it behaves different. Fun.
21:55
The same prompt might work perfectly on
21:57
one request and fail on the next. While
21:58
investigating, we also discovered that
22:00
the exact top K operation no longer had
22:03
prohibitive performance penalties that
22:04
it once did. We switched from
22:06
approximate to exact top K and
22:08
standardized some additional operations
22:09
on FP32 precision. Model quality is
22:12
non-negotiable. So we accepted the minor
22:14
efficiency impact. Happy that they're

22:16
finally putting this in words. I will
22:18
hold them to that comment. So why was
22:20
detection so difficult? Our validation
22:22
process ordinarily relies on benchmarks
22:24
alongside safety evals and performance
22:26
metrics. Engineering teams perform spot
22:29
checks and deploy to small canary groups
22:31
first. These issues exposed critical
22:33
gaps that we should have identified
22:34
earlier. Evaluations we ran simply
22:36
didn't capture the degradation that
22:38
users were reporting. In part because
22:39
Claude often recovers well from isolated
22:41
mistakes. Our own privacy practices also
22:44
created challenges in investigating
22:46
reports. Our internal privacy and
22:47
security controls limit how and when
22:49
engineers can access user interactions
22:51
with Claude. In particular, when those
22:53
interactions are not reported to us as
22:55
feedback, this protects user privacy,
22:57
but prevents engineers from examining

22:59
the problematic interactions needed to
23:01
identify or reproduce bugs. Privacy and
23:04
reliability are often at odds, and this
23:06
is a good example of that. That said,
23:08
they should have been able to catch it
23:09
internally. Each bug produced different
23:11
symptoms on different platforms at
23:13
different rates. This created a
23:14
confusing mix of reports that didn't
23:16
point to any single cause. It looked
23:18
like random inconsistent degradation.
23:21
That is also how it felt as a user. I
23:23
had so many weird problems with claw
23:24
during this window. More fundamentally,
23:27
we relied too heavily on noisy
23:29
evaluations. Although we were aware of
23:31
an increase in reports online, we lacked
23:33
a clear way to connect these to each of
23:35
our recent changes. When negative
23:37
reports spiked on August 29th, we didn't
23:39
immediately make the connection to an
23:40
otherwise standard load balancing

23:42
change. That one's a little harder to
23:44
forgive. So, let's see what they're
23:45
doing about this. As we continue to
23:48
improve our infra, we're also improving
23:49
the way that we evaluate and prevent
23:51
bugs like those discussed above across
23:53
all platforms where we serve Quad.
23:55
Here's what we're changing. Ignore the
23:57
cat. He has needs. Change one, more
24:00
sensitive evaluations. To help discover
24:02
the root cause of any given issue, we've
24:04
developed evaluations that can more
24:05
reliably differentiate between working
24:06
and broken implementations. We'll keep
24:08
improving these evaluations to keep a
24:10
closer eye on model quality. Change two
24:12
quality eval.
24:14
Although we run regular evals on our
24:16
systems, we will run them more
24:17
continuously on true prod systems to
24:19
catch issues such as the content window
24:21
loading balance error. Context window,

24:23
same difference. You get the idea. And
24:25
then change three, faster debugging
24:27
tooling. will develop infrastructure and
24:28
tooling to better debug community source
24:30
feedback without sacrificing user
24:32
privacy. Additionally, some bespoke
24:34
tools developed here will be used to
24:35
reduce remediation time in future
24:37
similar incidents if they should occur.
24:40
Also notice that nothing here mentions
24:42
refunds for affected customers. Over 30%
24:44
of cloud code users were affected and
24:46
approximately 0% of them got any money
24:48
back or anything back as a result. Evals
24:51
and monitoring are important, but these
24:53
incidents have shown that we also need
24:55
continuous signal from users when
24:56
responses from claude aren't up to the
24:58
usual standard. Reports of specific
25:00
changes observed, examples of unexpected
25:02
behaviors encountered, and patterns
25:04
across different use cases all helped us

25:05
isolate the issues. It remains
25:07
particularly helpful for users to
25:09
continue sending us their feedback
25:10
directly. You can use /bug as a command
25:12
in cloud code or use the thumbs down
25:14
button in the cloud apps to do so.
25:15
Developers and researchers often create
25:17
new and interesting ways to evaluate
25:18
models quality that complement our
25:20
internal testing. We like to share
25:21
yours. You can reach out to them
25:22
directly. There are things I would like
25:24
them to change. I wish that they were
25:27
more transparent as these things were
25:28
happening. I wish that they did this
25:30
type of posting more often. And most
25:32
importantly, I wish that they owned this
25:34
in a way where they were actually
25:35
refunding people or if they just like
25:37
extended everyone's claude code
25:38
subscription by two weeks as an apology.
25:40
There's a lot of things they could have

25:41
done here to own this better. And I am
25:43
surprised they didn't take those
25:45
opportunities, but at the same time,
25:47
Anthropic is as much an optics company
25:49
as they are a like actual tool we can
25:52
rely on. So, it's not surprising to me
25:54
that they went the optics route here.
25:56
All that said, this article represents a
25:58
significant change at Anthropic. This is
25:59
a level of transparency I just would
26:01
never have expected from them before.
26:02
I'm very thankful they're finally out
26:04
here doing it and I don't want to be too
26:06
mean because I really want to encourage
26:07
that they continue doing this. As we
26:09
build more and more things on top of the
26:11
non-deterministic, unreliable chaos that
26:13
is LLMs, it is more important than ever
26:16
that the companies providing those are
26:17
really transparent about what they're
26:19
providing and what went wrong with what
26:20
they're providing because none of this

26:22
is deterministic. We don't know what to
26:24
expect. So if things are less than what
26:27
is expected, they need to overcompensate
26:29
by doing more communication on top.
26:31
Anthropic historically hasn't done this
26:33
because they're still not used to being
26:34
a tool people rely on. They're really
26:36
just a research company at heart. And as
26:38
much as I love and respect that, we do
26:40
now rely on Anthropic as an info
26:42
company. And they need to step up to the
26:44
plate and own these things when they
26:46
happen. I got nothing else on this one.
26:47
I hope this was a useful breakdown. Let
26:49
me know what you guys think. And until
26:50
next time, he sns.